

Continual Self-Supervised Learning con Projected Functional Regularization: riproduzione e analisi

Papetti Greta

Progetto del corso di Streaming Data Analytics - E. Della Valle

1. Introduzione

Il progetto riproduce il paper *Continually Learning Self-Supervised Representations with Projected Functional Regularization* (Gomez-Villa et al., CVPR Workshop 2022), il cui codice pubblico è il repository `alviur/CVPR_PFR`. Il paper propone **PFR**, una tecnica di continual learning pensata per il caso self-supervised: una piccola rete di proiezione traduce le rappresentazioni correnti nello spazio di quelle precedenti, in modo da limitare il *catastrophic forgetting* senza conservare nessun dato dei task passati.

Il lavoro aveva tre obiettivi, che sono anche il filo di questo documento. Il primo è **mappare la teoria sul codice**: ritrovare nel repository i meccanismi descritti nel paper e collegarli ai concetti visti a lezione. Il secondo è **far girare del codice vecchio di quattro anni** su uno stack moderno, documentando cosa si rompe e perché. Il terzo è **riprodurre l'esperimento più semplice** — Barlow Twins su CIFAR-100 diviso in quattro task — e confrontare i numeri ottenuti con quelli del paper.

Il documento procede in tre blocchi: prima le nozioni del corso e il metodo (Sezioni 2 e 3), poi il lavoro tecnico — codice, patch, differenze rispetto al paper (Sezioni 4–6) — infine i numeri (Sezioni 7–9).

2. Le nozioni del corso

Questa sezione raccoglie tutto quello che serve sapere per leggere il resto del documento, nell'ordine in cui l'ho incontrato a lezione. Prima il continual learning e i suoi problemi, poi i tre scenari incrementali, poi il self-supervised learning, che nel corso non era il caso principale ma è quello su cui lavora il paper.

2.1. Continual learning: cosa cambia rispetto al machine learning classico

Il machine learning classico assume due cose: che tutta la conoscenza del problema stia in un training set finito, e che la distribuzione dei dati non cambi nel tempo. Nel continual learning cade la seconda assunzione. I dati arrivano a blocchi, e ogni blocco porta con sé una distribuzione nuova.

Il tipo di cambiamento, però, non è quello della parte *streaming* del corso, ed è la distinzione che vale la pena tenere ben ferma: **il drift del continual learning è additivo**. Se il modello ha imparato a riconoscere le cifre 0 e 1 e poi arrivano 2 e 3, il fatto che ora esistano 2 e 3 non rende sbagliato quello che sapeva prima. Il problema si estende, non si contraddice. Da qui una conseguenza pratica che si ritrova identica nel codice: **non serve un change detector**. Il cambio di distribuzione non va scoperto, è dato, ed è la struttura stessa dello stream — infatti l'indice del task è un parametro da riga di comando.

Lo stream è spezzato in **experience** $e_1, \dots, e_N$: ogni experience è un blocco di dati i.i.d. disponibile tutto insieme, e il **task** è il problema di classificazione associato. È proprio la disponibilità dell'intero blocco a rendere sensato allenarsi per cento epoche su ognuna, cosa che nello streaming vero e proprio non avrebbe senso.

Il nemico è il **catastrophic forgetting**: allenando in modo sequenziale e ingenuo, la rete sovrascrive i pesi utili alle experience passate per adattarsi a quella corrente, e le prestazioni sui vecchi test set crollano. Da qui il concetto cardine di tutto il resto, il **dilemma stabilità-plasticità**: troppa plasticità e si dimentica il passato, troppa stabilità e non si impara più niente di nuovo. Ogni strategia di continual learning è la scelta di un punto su questo asse.

Le strategie si dividono in tre famiglie: *replay-based*, che conservano campioni dei task vecchi e li rimescolano nel training; *architecture-based*, che espandono la rete a ogni task; *regularization-based*, che aggiungono alla loss un termine che vincola quanto il modello può cambiare. PFR appartiene alla terza ed è **exemplar-free**: non conserva nessun dato passato, che è il vincolo realistico quando ci sono ragioni di privacy o di spazio. L'unica traccia del passato è il modello stesso, com'era alla fine del task precedente.

2.2. I tre scenari incrementali

A lezione gli scenari incrementali sono tre, e si distinguono per due cose: cosa cambia da un'experience all'altra, e cosa il modello sa al momento del test.

Nell'**Incremental Task Learning** l'identità del task è nota anche in valutazione: il modello sa da quale blocco viene il campione e sceglie solo tra le classi di quel blocco, quindi ogni task ha il suo esame separato. Nell'**Incremental Domain Learning** le classi restano le stesse ma cambia il modo in cui si presentano — le stesse categorie fotografate, disegnate, viste con luce diversa — e l'identità del dominio non è nota. Nell'**Incremental Class Learning** arrivano classi nuove, l'identità del task non è nota, e il modello deve discriminare tra *tutte* le classi viste finora: è il caso più difficile, perché classi vecchie e nuove competono nella stessa decisione.

Questo progetto sta nel terzo scenario: CIFAR-100 diviso in quattro gruppi di 25 classi disgiunte, incontrati uno dopo l'altro. Lo scenario *domain* non è applicabile a CIFAR-100, dove ogni immagine appartiene a un solo stile visivo; il paper lo affronta su DomainNet, costruito apposta con più rappresentazioni della stessa categoria.

2.3. Self-supervised learning: imparare senza etichette

Il paper non lavora su un classificatore ma su un **estrattore di rappresentazioni**, ed è la differenza che cambia tutto il resto.

Un modello self-supervised non impara a dire *che cosa* è un'immagine: impara a *disporre* le immagini in uno spazio, in modo che quelle simili finiscano vicine. Il segnale di supervisione lo ricava dai dati stessi: si prende un'immagine, se ne creano due versioni applicando trasformazioni casuali (un ritaglio, un ribaltamento, un cambio di colore), e si chiede al modello di produrre per entrambe la stessa rappresentazione. Se due ritagli diversi della stessa foto devono dare lo stesso risultato, il modello è costretto a guardare il contenuto e non i dettagli superficiali.

Il metodo usato qui è **Barlow Twins**: le due viste passano nello stesso encoder e si costruisce la matrice di correlazione tra le dimensioni delle due rappresentazioni, che la loss spinge verso la matrice identità. I due termini hanno significati precisi. La *diagonale* portata a uno impone che le due viste diano la stessa rappresentazione, cioè l'invarianza alle trasformazioni; i termini *fuori diagonale* portati a zero impongono che dimensioni diverse siano scorrelate, cioè che ognuna dica qualcosa di nuovo. Senza il secondo termine il modello imboccherebbe la scorciatoia di rispondere sempre la stessa cosa a tutto: soddisfa il primo perfettamente ed è inutile.

A cosa serve una rappresentazione, se non classifica niente? Serve come base per qualunque compito venga dopo, e la si valuta con la **linear evaluation**: si congela l'encoder, gli si mette sopra un classificatore lineare e si allena solo quello. Se un classificatore così semplice separa le classi, l'informazione era già nella rappresentazione; e siccome l'encoder è congelato, ogni differenza tra due modelli dipende solo da come sono stati addestrati prima.

2.4. Mettere insieme le due cose

L'incrocio dei due mondi porta due conseguenze che nel corso non si presentano.

La prima riguarda **come si protegge il passato**. Nel continual learning supervisionato la tecnica classica è *Learning without Forgetting*: si tiene il modello vecchio e si chiede al nuovo di produrre le stesse *predizioni*. Qui le predizioni non esistono, perché durante il pre-training non c'è nessun classificatore. La distillazione va quindi spostata un piano più in basso, dalle predizioni alle **rappresentazioni**: è quello che fanno sia FD sia PFR.

La seconda riguarda **lo scenario**. Senza etichette e senza testa di classificazione, il modello non sa quante classi esistano né a quale task appartenga un'immagine, e non potrebbe usare l'informazione nemmeno volendo. Ne segue che qui **la distinzione tra i tre scenari non è osservabile durante il training**: si materializza solo in valutazione, ed è il protocollo di misura a decidere in quale dei tre ci si trova. Ci torno nella Sezione 7.

Tabella 1. Dalla lezione al progetto. Le nozioni sono raggruppate per come il paper si rapporta a quanto visto a lezione: se lo usa allo stesso modo, se lo sposta di piano, o se se ne discosta.

Nozione del corso	Come si presenta in questo progetto
CONVERGE <i>il concetto è usato esattamente come a lezione</i>	
Experience e_t	Quattro blocchi da 25 classi di CIFAR-100, con <code>--num_tasks 4</code>
Drift additivo	L'indice del task è passato da riga di comando: nel codice non esiste nessun rilevatore di drift
Stabilità-plasticità	È l'asse su cui si separano FT, FD e PFR, e il criterio con cui il paper stesso motiva PFR
Famiglie di strategie CL	PFR e FD ricadono nella terza famiglia: regularization-based, exemplar-free
Tuning degli iperparametri	Il problema aperto visto a lezione, qui in un caso concreto: λ tarato per un altro budget
Matrice R e metriche A , F , FWT	Costruite e calcolate con le formule delle slide
ESTENDE <i>il concetto regge, ma il self-supervised lo sposta di piano</i>	
Scenari incrementali	Senza etichette i tre scenari non sono distinguibili durante il training: lo scenario si decide in valutazione
Catastrophic forgetting	Va misurato sulle rappresentazioni e non sulle predizioni, e compare solo in alcuni protocolli
Learning without Forgetting	Stessa idea, ma la distillazione si sposta dalle predizioni alle rappresentazioni
DIVERGE <i>qui si fa qualcosa di diverso da quanto visto a lezione</i>	
Memoria M_t dell'algoritmo	Non contiene dati dei task passati: contiene un modello congelato, il <code>frozen_encoder</code>
Protocollo di valutazione	La lezione valuta un'experience alla volta, il paper in modo task-agnostic su tutte le classi

3. Il metodo: da Feature Distillation a PFR

Lo schema è sempre lo stesso: alla loss self-supervised del task corrente si aggiunge un termine che protegge il passato, e λ decide quanto il secondo pesa rispetto al primo. Quello che cambia da un metodo all'altro è *che cosa* il secondo termine protegge.

3.1. Feature distillation e il suo limite

La versione classica è la più diretta: si tiene una copia congelata del modello di fine task precedente — l'*insegnante* — e si chiede al modello in addestramento — lo *studente* — di produrre per le stesse immagini rappresentazioni vicine a quelle dell'insegnante.

Il problema, che è il bersaglio polemico del paper, è che così si danno al modello due ordini in contraddizione: *impara cose nuove* e *non cambiare*. La soluzione perfetta del secondo è non muoversi affatto, e più si alza λ più quella soluzione conviene. Ne esce un modello stabile e poco plastico, quella che il paper chiama *intransigenza*: l'incapacità di imparare il task nuovo.

3.2. L'idea di PFR

PFR parte da un'osservazione più fine: il modello nuovo non deve **dire** la stessa cosa del vecchio, deve **sapere** la stessa cosa. È una richiesta molto più debole, e per imporla serve un modo di verificare che l'informazione ci sia ancora anche se è espressa diversamente.

Il modo è la traduzione. Il modello vecchio parla una lingua, il nuovo se ne costruisce un'altra, più ricca perché ha visto più dati. Per verificare che la seconda non abbia perso niente della prima si cerca un **dizionario** che riporti dalla nuova alla vecchia: se esiste, tutto ciò che c'era prima è ancora recuperabile. PFR è questo, e il dizionario è una piccola rete m allenabile. Il vincolo non è più «parla la lingua vecchia», ma «parla la lingua che vuoi, purché esista un dizionario che riporta a quella vecchia».

3.3. L'equazione, pezzo per pezzo

Formalmente, al task t si minimizza:

$$\underbrace{\mathcal{L}_c^t}_{(1)} + \lambda_{pfr} \mathbb{E}_x \left[\underbrace{\mathcal{S} \left(\underbrace{m(f_{\theta^t}(x))}_{(2)}, \underbrace{f_{\theta^{t-1}}(x)}_{(3)} \right)}_{(4)} \right]$$

Letta pezzo per pezzo:

- **(1)** è la loss self-supervised del task corrente, cioè Barlow Twins sulle immagini che sto vedendo adesso. È il termine che spinge a imparare.
- **(3)** è la rappresentazione prodotta dal modello **congelato** del task precedente sulla *stessa* immagine di adesso. È l'unica traccia rimasta del passato: i vecchi dati non ci sono più, ma il vecchio modello può ancora essere interrogato.
- **(2)** è la rappresentazione prodotta dal modello attuale, *passata attraverso il dizionario m* . Questo è il contributo del paper: non si confronta la feature nuova con quella vecchia, si confronta la sua **traduzione**.
- **(4)** è la misura di somiglianza tra le due, la cosine similarity, che vale uno quando i due vettori puntano nella stessa direzione.
- λ_{pfr} decide quanto il secondo termine pesa rispetto al primo. È la manopola tra stabilità e plasticità, e nella Sezione 6 si vedrà che è anche la fonte di quasi tutti i problemi.

Due dettagli fanno funzionare il meccanismo. Il primo è che **m si allena insieme al modello**: insegue le riorganizzazioni dello spazio per continuare a tradurre. Ne nasce una divisione del lavoro — la parte di cambiamento che m assorbe non costa niente all'encoder, penalizzato solo per ciò che il dizionario non riesce a coprire. È la differenza di fondo con la feature distillation, dove ogni cambiamento viene pagato.

Il secondo è che **m è una funzione sola, la stessa per tutte le immagini**. Se potesse comportarsi diversamente caso per caso, un dizionario abbastanza furbo giustificerebbe qualunque scempio e il vincolo non vincolerebbe niente. Dovendo valere per tutte le immagini insieme, può assorbire solo i cambiamenti *sistematici*, non gli aggiustamenti su misura.

4. Dove tutto questo sta nel codice

Questa sezione risponde a cinque domande concrete: come si combinano il metodo self-supervised e la strategia continual, dove sta la memoria del passato, come è scritto PFR, in che cosa la feature distillation differisce da PFR, e come nascono le quattro experience. Per ognuna riporto le righe che contano, semplificate solo nella formattazione.

4.1. Come si combinano metodo e strategia

Il repository tiene separate due cose che di solito si trovano mescolate: il metodo self-supervised (cosa il modello impara) e la strategia di continual learning (come viene protetto il passato). Si compongono in tre righe:

```
# main_pretrain.py
MethodClass = METHODS[args.method] # es. barlow_twins
if args.distiller:
    MethodClass = DISTILLERS[args.distiller](MethodClass) # avvolge il metodo
model = MethodClass(**args.__dict__)
```

METHODS e DISTILLERS sono due dizionari che mappano il nome scritto da riga di comando alla classe corrispondente. La prima riga sceglie il metodo self-supervised. La seconda, se ho passato `--distiller`, non lo sostituisce: lo **avvolge**. `DISTILLERS["pfr"]` è una funzione che prende una classe e ne restituisce una nuova, identica ma con in più la copia congelata del passato e il termine di regolarizzazione nella loss.

Due conseguenze pratiche, che sono anche il motivo per cui il confronto sperimentale di questo progetto è pulito:

- **Il fine-tuning ingenuo non è un metodo a parte: è l'assenza del secondo blocco.** Se non passo `--distiller`, il modello è Barlow Twins puro, allenato in sequenza. Tra FT, FD e PFR cambia quindi una cosa sola.
- **Il distiller non entra mai in valutazione.** `main_linear.py` carica dal checkpoint soltanto l'encoder, quindi tre modelli addestrati in modi diversi vengono poi esaminati in modo identico.

4.2. Dove sta la memoria del passato

Trattandosi di un metodo exemplar-free, i dati vecchi non esistono più: l'unica traccia del passato è il modello di fine task precedente. Il codice ne fa una copia all'inizio di ogni task e la disattiva:

```
# cassle/distillers/base.py
def on_train_start(self):
    if self.current_task_idx > 0:
        self.frozen_encoder = deepcopy(self.encoder) # fotografia del modello attuale
        for pg in self.frozen_encoder.parameters():
            pg.requires_grad = False # non verra' piu' aggiornata

    @torch.no_grad()
    def frozen_forward(self, X):
        return self.frozen_encoder(X) # si puo' solo interrogare
```

`deepcopy` crea una copia indipendente della rete, che da quel momento non cambia più: `requires_grad = False` le toglie il diritto di essere aggiornata, e il decoratore `@torch.no_grad()` evita perfino di calcolarne i gradienti. Questa copia è **l'insegnante**: le si mostrano le immagini nuove e si guarda cosa risponde. È la memoria M_i che la formalizzazione vista a lezione mette accanto al modello — con la differenza che qui non contiene dati, contiene un modello. Viene salvata anche nel checkpoint, tanto che ogni file `.ckpt` di FD e PFR contiene due reti invece di una.

4.3. PFR

Il metodo del paper sta tutto in una decina di righe. Prima si costruisce la rete di traduzione:

```
# cassle/distillers/pfr.py -- costruzione
self.distill_predictor = nn.Sequential( # la rete m
    nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Linear(256, 512))
```

È un piccolo perceptron a due strati che prende un vettore di 512 numeri, lo comprime a 256 e lo riporta a 512. Il restringimento centrale non è un dettaglio: è ciò che impedisce a m di imparare la funzione identità e di rendere il vincolo banale.

Poi, a ogni batch di immagini:

```
# cassle/distillers/pfr.py -- passo di training
f1 = self.encoder(x1) # rappresentazione del modello attuale
frozen_f1 = self.frozen_forward(x1) # rappresentazione dell'insegnante

p1 = self.distill_predictor(f1) # traduco la rappresentazione nuova
distill_loss = -cosine_similarity(p1, frozen_f1.detach()).mean()

return out["loss"] + self.distill_lambda * distill_loss
```

Riga per riga: `f1` è come il modello di adesso vede l'immagine, `frozen_f1` è come la vedeva alla fine del task precedente. `p1` è la *traduzione* della prima nel linguaggio della seconda, ed è il contributo del paper — non si confrontano `f1` e `frozen_f1` direttamente, si confronta `p1` con `frozen_f1`. La `cosine_similarity` misura quanto i due vettori puntano nella stessa direzione e vale 1 quando coincidono; il segno meno la trasforma in una quantità da minimizzare. Il `.detach()` taglia il collegamento verso l'insegnante, così l'ottimizzatore non prova ad aggiustare *lui* per far tornare i conti. Infine `out["loss"]` è la loss di Barlow Twins sul task corrente, e `distill_lambda` è il λ che decide quanto il vincolo pesa rispetto all'apprendimento.

4.4. La feature distillation, e in che cosa differisce

La baseline ha esattamente la stessa forma:

```
# cassle/distillers/decorrelative.py
p1 = self.distill_predictor(z1)          # z1, non f1
distill_loss = barlow_loss_func(p1, frozen_z1) # righello diverso
return out["loss"] + self.distill_lamb * distill_loss # distill_lamb = 25
```

Cambiano però due cose, ed è importante perché nel report FD e PFR vengono confrontate di continuo. La prima è **su cosa si applica il vincolo**: $z1$ è l'uscita del *projector*, la piccola testa che Barlow Twins usa per calcolare la propria loss, mentre $f1$ è l'uscita del *backbone*, cioè la rappresentazione vera e propria — quella che poi verrà valutata. La seconda è **con quale righello** si misura la somiglianza: PFR usa la cosine similarity, FD usa la stessa funzione della loss di Barlow Twins.

Ne segue una conseguenza che torna nella Sezione 6: i due λ moltiplicano quantità di natura e scala diverse, quindi **non sono confrontabili tra loro** — 25 per FD non significa «venticinque volte più severo» di 1 per PFR.

4.5. Come nascono le quattro experience

Lo stream non è un flusso continuo, è un filtro sulle etichette applicato al dataset:

```
# cassle/utils/pretrain_dataloader.py
if split_strategy == "class":
    mask = [(c in tasks[task_idx]) for c in dataset.targets]
    task_dataset = Subset(dataset, torch.tensor(mask).nonzero())
```

`tasks` è la lista dei gruppi di classi — nel mio caso quattro gruppi da 25, ottenuti mescolando le 100 classi di CIFAR-100 con un seed fisso — e `task_idx` dice quale gruppo tocca adesso. Il risultato è un sottoinsieme del dataset che contiene solo quelle classi.

Due osservazioni. La prima è che l'argomento `--split_strategy` accetta tre valori, `class`, `data` e `domain`: è la tassonomia degli scenari della Sezione 2.2 trasformata in un parametro da riga di comando. Il ramo `domain`, quello degli esperimenti su DomainNet, nel codice pubblico solleva `NotImplementedError`. La seconda è che le etichette compaiono *solo qui*, per decidere cosa entra in ogni blocco: durante il training non vengono mai usate, ed è per questo che il modello non ha modo di sapere in quale scenario si trovi.

5. Far rivivere il codice

Il repository è del 2022, scritto per PyTorch 1.10 e PyTorch Lightning 1.5.4, e su un ambiente moderno non parte. Ho scelto di **congelare le versioni delle librerie invece di modernizzare il codice**: portare tutto a Lightning 2.x avrebbe significato riscrivere il training loop, cioè proprio la parte che devo riprodurre fedelmente. Con le versioni originali servono tre patch, tutte minime.

La prima ricrea il modulo `torch._six`, che PyTorch 2.0 ha rimosso ma che Lightning 1.5.4 importa ancora. La seconda toglie un parametro deprecato (`compute_on_step`) dalla chiamata a `torchmetrics` dentro `cassle/utils/knn.py`. La terza sostituisce, dentro Lightning, un controllo di tipo sullo scheduler del learning rate: nelle versioni recenti di PyTorch alcuni scheduler non ereditano più dalla classe che Lightning si aspetta, e il trainer li rifiuta.

Il punto importante è che **nessuna delle tre tocca la logica del metodo**: due sono import e una è un controllo di tipo, e nessuna passa vicino ai distiller o alle loss. Questo è ciò che rende difendibile l'affermazione che quello che ho eseguito è ancora il codice degli autori. Che le patch non abbiano alterato i numeri non l'ho comunque dato per scontato: nella Sezione 8 riscrivo la loss di Barlow Twins partendo dalla formula e la confronto con quella del repository sugli stessi input.

6. Paper, repository e mia esecuzione: le differenze

Prima di guardare i risultati serve mettere in chiaro rispetto a *che cosa* li sto confrontando, perché le fonti non sono due ma tre: il testo del paper, gli script pubblicati dagli autori, e la mia esecuzione. E le prime due non coincidono tra loro.

6.1. Il repository è un fork di un altro progetto

Il codice di PFR non è stato scritto da zero: è un fork di **CaSSLe**, un framework per il continual self-supervised learning di un altro gruppo. Gli autori hanno aggiunto il proprio distiller (`pfr.py`), ritoccato i parametri negli script di lancio e lasciato invariato il resto — negli script è rimasto persino il nome del progetto di tracciamento del laboratorio originale. Questo spiega gran parte dei disallineamenti: il testo descrive l'idea nella sua forma pulita, il codice riflette il framework realmente usato, con i default ereditati da qualcun altro.

Il caso più evidente è la baseline. Il paper definisce la feature distillation come **distanza L2** tra le rappresentazioni; lo script degli autori per CIFAR usa invece il distiller `decorrelative`, ereditato da CaSSLe, che misura la somiglianza con la stessa funzione della loss di Barlow Twins. Il repository *contiene* anche un distiller L2 (`predictive_mse.py`), ma non è quello che gli script lanciano. Ho scelto di riprodurre ciò che fa il codice, non ciò che dice il testo, e di segnalare la differenza.

6.2. La questione di λ

La differenza più vistosa riguarda il peso della regolarizzazione, e a prima vista sembra un errore clamoroso. Nel paper, la Figura 4b esplora la feature distillation con λ tra **0.01 e 0.1**; lo script degli autori usa **25**. Tre ordini di grandezza.

Non è però una contraddizione, ed è il punto che mi ha richiesto più tempo per essere chiarito. λ **non ha unità di misura**: è una manopola di bilanciamento, e il suo valore non significa niente se non si sa che cosa moltiplica. Le due FD moltiplicano cose diverse — la L2 del paper produce numeri grandi rispetto alla loss del task e va tenuta bassa, la loss stile Barlow del codice produce numeri dello stesso ordine e per pesare davvero va alzata. È come dire che una strada è lunga «dieci metri» o «0.01 chilometri».

Tabella 2. Le due feature distillation a confronto: stesso nome, loss diverse, quindi λ su scale diverse.

	Paper (eq. 4 e Fig. 4b)	Codice (bash file per CIFAR)
Che cos'è la FD	La feature distillation classica: distanza L2 tra le rappresentazioni	Il distiller decorrelative, ereditato dal framework CaSSLe
Che cosa moltiplica λ	Una distanza L2, che produce numeri grandi rispetto alla loss del task	La stessa loss di Barlow Twins, che produce numeri piccoli
Quanto vale λ	Va tenuto basso: da 0.01 a 0.1	Va alzata: 25

Restano però due fatti che vale la pena registrare. Il primo è che $\lambda = 25$ è una scelta **deliberata e specifica per CIFAR**: negli script per gli altri dataset il parametro non viene passato affatto, quindi resta al default di 1, e nel repository originale di CaSSLe non c'era. Il secondo è che gli autori hanno tarato quel valore per **500 epoche per task**, mentre io ne ho a disposizione 100 — e un valore giusto in un regime non è detto che lo sia in un altro. È da qui che parte l'indagine della Sezione 8.

6.3. Il quadro completo

La Tabella 3 mette a confronto le tre fonti. Vale la pena notare che su alcune voci la mia scelta segue il paper e su altre segue il codice: dove le due divergevano ho scelto di volta in volta, dichiarando il motivo.

Tabella 3. Paper, script degli autori e mia esecuzione a confronto.

Voce	Paper (testo)	Script autori	Mia esecuzione
Epoche per task	1500	500	100 (vincolo di calcolo: è la semplificazione che pesa di più)
Numero di task	4	5	4 , come il paper
Classi per task	25	20	25 , come il paper
FD implementata come	Distanza L2	decorrelative	decorrelative, come gli script
λ per FD	0.01 – 0.1 (su L2)	25 (su Barlow)	25, come gli script; più una prova a 5
λ per PFR	2 – 100	non passato $\rightarrow$ 1	1, come gli script
Learning rate	0.06	0.3 con LARS	0.3, come gli script
Valutazione	Task-agnostic (100 classi)	Solo probe a 100 classi	Tre protocolli, più il kNN
Ripetizioni	Media su più run	—	Un solo seed

Un disallineamento del genere è normale nella riproduzione di lavori di machine learning e non è di per sé un difetto del paper: gli script pubblicati sono esempi rappresentativi, non la griglia esatta con cui è stata prodotta ogni tabella. L'approccio che ho seguito è **riprodurre quello che fa davvero il codice e dichiarare le divergenze**, senza assumere che una delle due fonti sia la verità.

7. Risultati

Ho confrontato quattro condizioni: **FT**, fine-tuning sequenziale senza protezione (riferimento in basso); **FD**, la feature distillation del repository; **PFR**, il metodo del paper; **Joint**, il training su tutte le 100 classi insieme (riferimento in alto). Il task iniziale è allenato una volta sola e condiviso dai tre metodi continual, così partono tutti dallo stesso punto.

Da ogni sequenza di checkpoint costruisco la **matrice R**: $R[t, i]$ è l'accuracy sull'experience i dopo aver allenato fino a t . La diagonale dice quanto bene si impara, sotto la diagonale quanto si ricorda, sopra come ci si comporta su ciò che non si è ancora visto. Da lì le metriche del corso: l'**accuracy finale** (media dell'ultima riga), il **backward transfer** (negativo quando c'è forgetting) e il **forward transfer**.

7.1. Perché tre protocolli e due strumenti

Come anticipato, in un contesto self-supervised è il protocollo di valutazione a determinare lo scenario. La domanda che lo decide è una sola: **quando classifico un campione di test, tra quali classi lo faccio scegliere?** Tre risposte possibili, che misurano cose diverse.

- **Task-Aware.** So a priori da quale task viene il campione, e la scelta avviene solo tra le 25 classi di quel task. È la valutazione *task-incremental*: l'identità del task è fornita come oracolo. Misura la qualità delle rappresentazioni task per task, eliminando ogni possibilità di confusione tra classi di task diversi. È un limite superiore ottimistico.
- **Seen-Pooled.** Non so da quale task venga il campione, e la scelta avviene tra tutte le classi *viste finora*: 25 dopo la prima experience, poi 50, poi 75, poi 100. È la valutazione *class-incremental* vera e propria, quella realistica. La matrice è triangolare, perché non ha senso testare su classi che il modello non ha ancora incontrato.
- **All-Pooled.** La scelta avviene sempre tra tutte le 100 classi, comprese quelle di task futuri. È il protocollo di `main_linear.py`, cioè quello usato dal paper. Misura la qualità assoluta dello spazio delle rappresentazioni, indipendentemente dall'ordine con cui i task sono arrivati.

Delle tre, il repository offriva solo la All-Pooled; le altre due le ho implementate io. Ho poi ripetuto tutto con due strumenti diversi. Il **probe lineare** è un classificatore addestrato sopra l'encoder congelato: traccia confini globali e va allenato. La **valutazione kNN** non allena niente: per classificare un'immagine di test cerca le venti immagini di training più simili nello spazio delle rappresentazioni e le fa votare. Il motivo di usarli entrambi è che il probe è pur sempre un modello addestrato, e si potrebbe obiettare che il risultato dipenda da quanto bene è stato allenato lui; il kNN toglie l'obiezione, perché non ha parametri né scelte di ottimizzazione. Se due strumenti così diversi raccontano la stessa storia, la conclusione non dipende dallo strumento.

Tabella 4. Le tre metriche del protocollo di valutazione nei tre scenari, con entrambi gli strumenti. **A** = accuracy finale, **BWT** = backward transfer, **FWT** = forward transfer. In grassetto i BWT negativi, cioè gli unici casi in cui il forgetting si manifesta. Il FWT non è definito nella seen-pooled, dove la matrice è triangolare e non esistono elementi sopra la diagonale; il Joint, avendo un solo checkpoint, ha solo l'accuracy. (†) valore ottenuto con `main_linear.py`, cioè l'implementazione del repository dello stesso protocollo.

Protocollo	Strum.	FT (No CL)			FD			PFR			Joint
		A	BWT	FWT	A	BWT	FWT	A	BWT	FWT	
Task-Aware	Probe kNN	74.5	+3.9	62.2	59.5	+0.1	54.3	74.2	+3.2	62.1	78.7
		68.8	+2.9	54.2	53.6	+0.5	46.9	68.2	+2.7	54.4	—
Seen-Pooled	Probe kNN	56.2	-6.2	—	41.5	-10.1	—	56.7	-6.0	—	—
		49.3	-4.5	—	34.5	-9.8	—	49.3	-4.4	—	—
All-Pooled	Probe kNN	56.5	+4.1	47.6	43.3	+0.6	40.7	57.6	+5.0	47.4	59.0 [†]
		49.3	+6.6	38.3	34.5	+1.2	31.7	49.3	+6.8	38.5	—

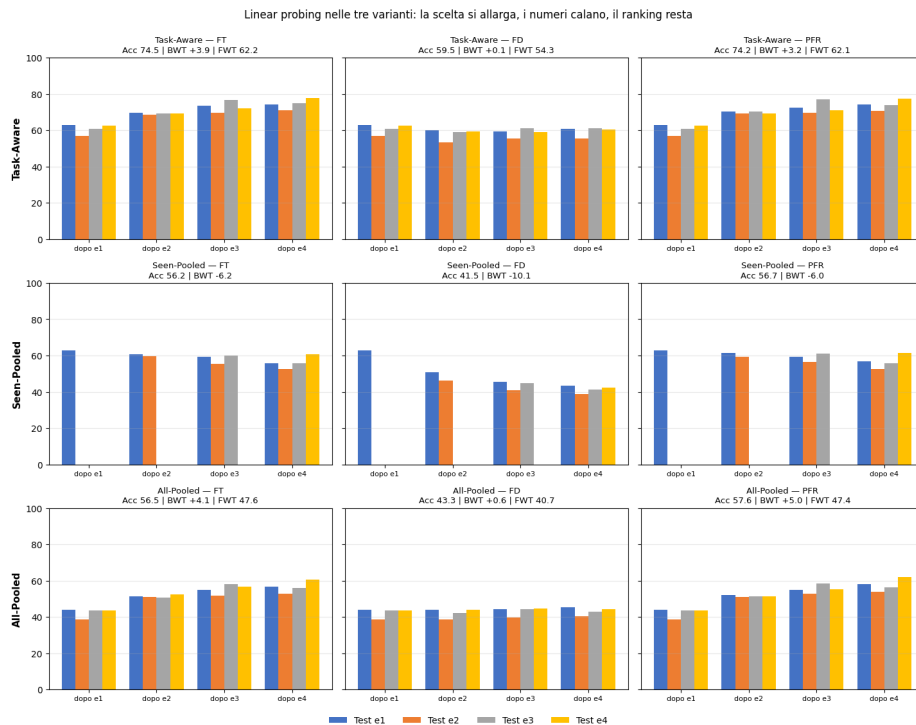


Figura 1. Probe lineare nelle tre varianti. Ogni gruppo di barre è una riga della matrice *R*: dopo il training fino a quell'experience, l'accuracy su ciascuna delle quattro. Nella seen-pooled mancano le barre dei task futuri, non ancora valutabili.

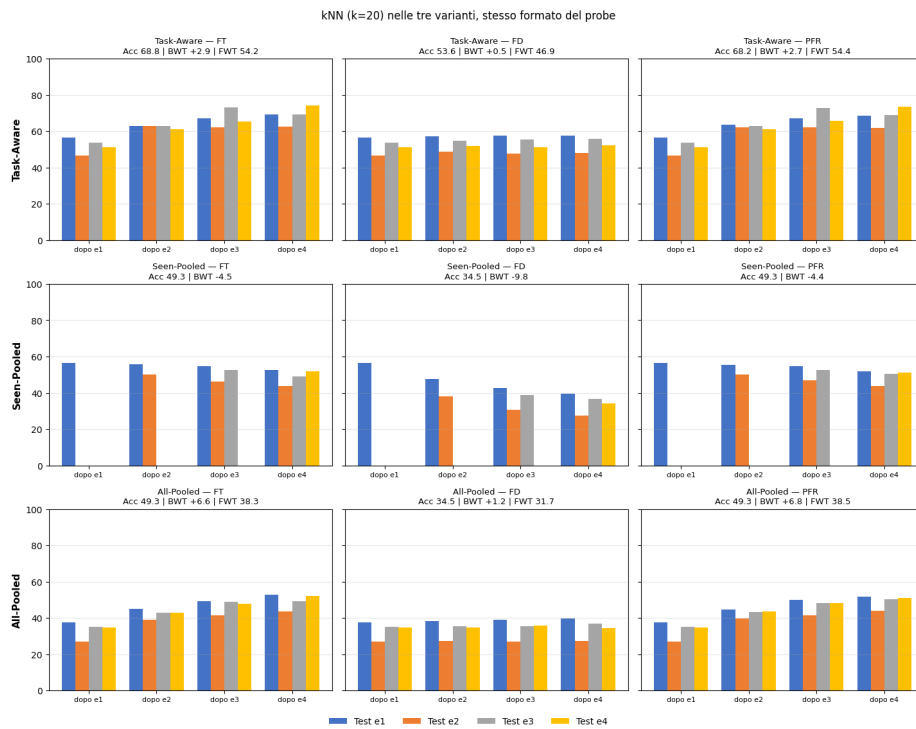


Figura 2. Valutazione kNN nelle tre varianti, stesso formato e stessi checkpoint. Tutti i valori sono più bassi di quelli del probe, ma la forma delle barre e l'ordine tra i metodi sono gli stessi.

7.2. Probe e kNN: uno scarto costante

La prima cosa da guardare è la differenza tra i due strumenti, perché se fosse irregolare non potrei fidarmi di nessuno dei due. Il kNN è **sistematicamente più basso del probe**, ma di una quantità stabile: circa 6 punti nella task-aware (74.5 contro 68.8 per FT, 59.5 contro 53.6 per FD, 74.2 contro 68.2 per PFR), circa 7 nella seen-pooled, circa 8 nella all-pooled. Lo scarto è quindi quasi identico per tutti e tre i metodi dentro ciascun protocollo.

Il motivo è che i due strumenti guardano la stessa cosa in modi diversi: il probe traccia confini globali e può sfruttare direzioni utili anche se le classi non sono raggruppate in modo compatto, il kNN guarda solo le distanze locali e quindi chiede di più alla rappresentazione. Il punto è che questa differenza è **un'inclinazione uniforme, non una distorsione**: sposta tutti verso il basso della stessa quantità e non cambia mai né l'ordine dei metodi né il segno del backward transfer. È il controllo che volevo — la storia raccontata dai numeri non dipende dallo strumento di misura.

7.3. I tre protocolli: dove si nasconde il forgetting

Il confronto tra protocolli è molto più interessante, perché lì i numeri non si spostano soltanto: cambiano di segno.

Passando dalla task-aware alle due pooled si perdono circa 18 punti. FT con il probe passa da 74.5 a 56.2, PFR da 74.2 a 56.7, FD da 59.5 a 41.5. Il calo è di nuovo praticamente uguale per tutti, ed è atteso: scegliere tra 25 alternative è molto più facile che scegliere tra 100. Questa parte del divario non ha niente a che vedere col continual learning, è solo la difficoltà del compito.

Ma il backward transfer non si sposta: si ribalta. Ed è il risultato che trovo più istruttivo di tutto il progetto. Con la valutazione task-aware, FT ha BWT +3.9: i task vecchi *migliorano* mentre il modello impara i nuovi. Con la seen-pooled, lo stesso identico modello, valutato sulle stesse identiche rappresentazioni, ha BWT -6.2. E il kNN dice la stessa cosa, +2.9 contro -4.5.

La spiegazione è che i due protocolli misurano due fenomeni diversi. La task-aware chiede: *le classi del primo task sono ancora distinguibili tra loro?* La risposta è sì, anzi sempre di più, perché continuando ad allenarsi il modello costruisce rappresentazioni migliori per tutti. La seen-pooled chiede invece: *le classi del primo task sono ancora distinguibili da tutte le altre?* E qui la risposta peggiora a ogni experience, perché ogni gruppo di classi nuove aggiunge candidati con cui i vecchi campioni possono essere confusi. Le barre della Figura 1, riga centrale, mostrano proprio questo: la prima barra di FT scende da circa 63 a 61, poi 59, poi 56, man mano che la galleria si allarga.

Ne segue una conclusione che vale la pena mettere per iscritto: **buona parte di quello che chiamiamo forgetting in uno scenario class-incremental è interferenza tra classi, non perdita di informazione** — e una valutazione task-aware non la vede affatto. È anche la ragione per cui la All-Pooled, che è il protocollo del paper, ha BWT positivo come la task-aware: usando fin dall'inizio tutte le 100 classi, la difficoltà del compito non aumenta nel tempo, e l'interferenza c'è ma è costante invece di crescere. Solo la seen-pooled fa crescere la difficoltà insieme allo stream, ed è per questo che è l'unica in cui il forgetting emerge.

Un dettaglio che conferma la coerenza dei conti: nell'ultima riga della matrice le due varianti pooled coincidono per costruzione, perché dopo la quarta experience «tutte le classi viste» e «tutte le classi» sono la stessa cosa. Con il kNN infatti

l'accuracy finale è identica (49.3 in entrambe per FT); con il probe differisce di pochi decimi, perché il classificatore delle due varianti viene addestrato su insiemi di classi diversi.

7.4. Il confronto con il paper

Al netto del protocollo, il quadro è chiaro: **FT e PFR sono appaiati, FD sta quindici punti sotto entrambi**. Il vantaggio di PFR c'è ma è minimo, e compare solo nei protocolli pooled — nella seen-pooled è avanti di mezzo punto (56.7 contro 56.2), nella all-pooled di un punto (57.6 contro 56.5), mentre nella task-aware è FT a essere avanti di tre decimi. Sono differenze al limite di ciò che un singolo seed può affermare, ma il segno è coerente: dove il forgetting si manifesta, PFR è leggermente meglio.

La Tabella 5 mette i miei numeri accanto a quelli del paper, nelle tre condizioni che il progetto chiede di confrontare: il training offline come limite superiore, il fine-tuning senza protezione come riferimento in basso, e la tecnica di continual learning nel mezzo. Uso qui il protocollo del paper, cioè il probe a 100 classi di `main_linear.py`, che è l'unico in cui il confronto è legittimo.

Tabella 5. Confronto con il paper, nel protocollo del paper (probe a 100 classi con `main_linear.py`). **A** = accuracy finale, **F** = forgetting, **FWT** = forward transfer. Il paper riporta soltanto l'accuracy, quindi per F e FWT non c'è un termine di paragone; il suo limite superiore è 60.6% (il training offline su tutti i dati arriva a circa 63%).

Condizione	Metodo	A	F	FWT	A (paper)
Full training (upper bound)	Joint	59.0	—	—	60.6
No CL (fine-tuning)	FT	54.0	0.91	41.9	56.8
Tecnica CL (baseline)	FD	41.1	0.19	34.0	57.2
Tecnica CL (paper)	PFR	54.2	1.28	41.2	60.1

Il confronto è **negativo**. Nel paper l'ordine è crescente — FT sotto, FD in mezzo, PFR sopra, con un vantaggio netto di quest'ultimo — mentre da me FT e PFR sono appaiati e FD sprofonda. Anche il divario dal limite superiore si comporta diversamente: nel paper PFR arriva praticamente a toccarlo (60.1 contro 60.6), da me resta cinque punti sotto, esattamente come il fine-tuning.

Vale la pena notare la colonna del forgetting, perché è il caso da manuale di una metrica che va letta insieme alle altre: il valore più basso di tutti, 0.19, è quello di FD — cioè del metodo che ha imparato di meno. Non dimentica perché non ha acquisito niente da dimenticare. Le due anomalie hanno comunque cause diverse.

La prima — FT che non dimentica — è **un effetto del budget di epoche**. A 100 epoche il modello è lontano dalla convergenza: le rappresentazioni sono ancora generiche, valgono un po' per tutte le classi, e qualunque dato nuovo le migliora, anche per i task vecchi. Il forgetting da manuale richiede che il modello si sia davvero specializzato su un task, cosa che nel regime del paper succede e qui no. E se non c'è forgetting da prevenire, PFR non ha niente da correggere: non fa danni, ma non ha modo di aiutare. È la frase vista a lezione — *avoiding forgetting slows down the plasticity* — letta al contrario: quando la bilancia ha un piatto vuoto, proteggere non serve.

La seconda anomalia, il crollo di FD, meritava un'indagine vera.

8. L'indagine: perché FD va così male

Si trattava di capire se il problema fosse un errore mio, un bug del codice o una conseguenza prevedibile della configurazione. Ho proceduto per esclusione, restringendo il campo a ogni passo.

1. **È colpa dell'esame?** No. Il confronto tra i tre protocolli e i due strumenti della sezione precedente mostra che cambiando esame tutti i metodi si spostano più o meno allo stesso modo, il ranking non cambia mai, e le barre di FD restano piatte in tutte e sei le combinazioni. La misura è assolta: il problema è nel modello.
2. **Quando smette di imparare?** Le curve per epoca (Figura 3) mostrano il sintomo in diretta: FD resta una decina di punti sotto gli altri per tutto il training, su ogni task. Se il problema fosse un eccesso di protezione ci si aspetterebbe almeno di vederla *migliore* sulle classi vecchie — invece è peggiore ovunque, anche lì. Guardando separatamente le due componenti della loss si trova l'altra metà del quadro: quella di distillazione si appiattisce quasi subito, cioè il vincolo viene soddisfatto immediatamente, mentre quella self-supervised scende molto più lentamente che in PFR. FD sta spendendo il suo budget per assomigliare all'insegnante, non per imparare il task nuovo.

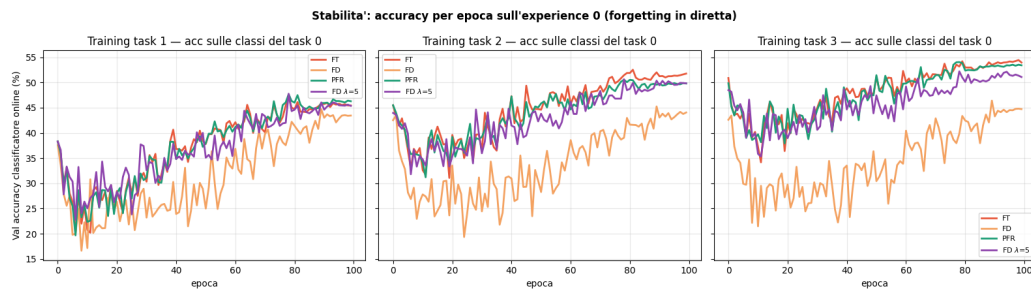


Figura 3. Accuracy sulle classi della prima experience mentre il modello impara le successive. Tutte le curve scendono all'inizio di ogni task e poi risalgono; quella di FD (arancione) resta stabilmente più in basso.

3. I pesi si sono mossi? Il frozen_encoder è identico bit per bit all'encoder di fine task precedente, quindi la memoria funziona come deve; misurando quanto ogni metodo si è allontanato da quel riferimento, FD risulta praticamente fermo. Con un dettaglio curioso: i suoi pesi non sono immobili — la loro scala è più che raddoppiata — ma la *direzione* è cambiata pochissimo. È una rete che si muove molto, sempre lungo la stessa linea, senza mai riorganizzare lo spazio. Il perché viene fuori al punto 6.

4. Come appaiono le rappresentazioni? Prendendo cinque classi della seconda experience e proiettandole in due dimensioni (Figura 4): il modello di partenza le tiene mescolate, perché non le ha mai viste; FT e PFR le separano in gruppi riconoscibili; FD le lascia quasi come le ha trovate. Non ha imparato e poi dimenticato — non ha proprio imparato.

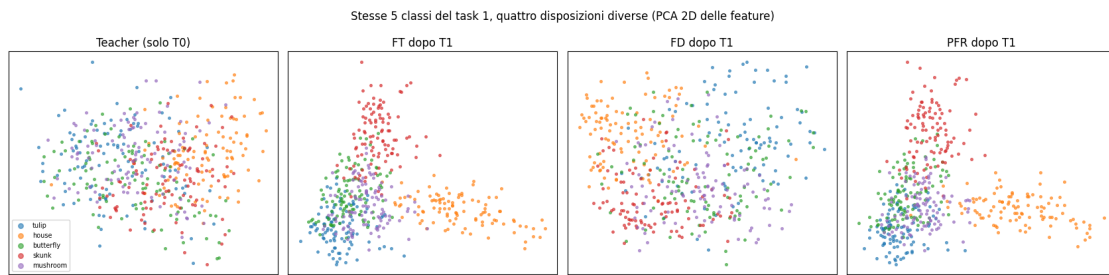


Figura 4. Le stesse cinque classi della seconda experience, viste da quattro modelli diversi (proiezione in due dimensioni). FT e PFR le separano, FD resta vicinissimo al punto di partenza.

5. È colpa del codice, o della backpropagation? Prima di accusare la configurazione bisogna escludere il codice, e in particolare un tipo di bug che darebbe esattamente questo sintomo. Dopo aver calcolato la loss, PyTorch percorre la rete **all'indietro** e calcola per ogni peso il *gradiente*: quanto ha contribuito all'errore e in che direzione andrebbe corretto. Questo percorso a ritroso è la **backpropagation**. Basta un `detach()` nel posto sbagliato e il flusso si interrompe: un pezzo di rete **smette di imparare senza dare nessun errore**, che è precisamente il comportamento di FD. Ho ricostruito la situazione in piccolo e fatto quattro verifiche:

- *L'insegnante è davvero congelato?* Nessuno dei suoi parametri richiede gradiente, e dopo tutti i passaggi all'indietro non ne ha ricevuto nessuno.
- *La loss del task raggiunge l'encoder?* Sì, con gradiente diverso da zero. È il valore di riferimento.
- *La distillazione raggiunge l'encoder?* Anche sì — e va notato che il bug temuto avrebbe reso FD *identica* a FT, non peggiore.
- *Le librerie calcolano i numeri giusti?* Ho riscritto la loss di Barlow Twins dalla formula e l'ho confrontata con quella del repository sugli stessi input: differenza dell'ordine di 10^{-4} , cioè solo arrotondamento. Le tre patch sono assolute.

Il codice è innocente. Ma la terza verifica, oltre a rispondere sì, misura anche *con che forza* il vincolo tira — ed è lì che si trova la risposta.

6. Con che forza tira il vincolo? Il gradiente che il termine di distillazione produce sull'encoder è circa **250 volte più grande** di quello prodotto dalla loss del task. A ogni passo i pesi si muovono nella somma dei due contributi, e quando uno è 250 volte l'altro la somma punta praticamente dove punta il più grande. Il modello passa il suo tempo a obbedire al vincolo, non a imparare — la stessa cosa che si vedeva nelle curve del punto 2.

6a. Perché allora il training non esplode? Un rapporto del genere farebbe pensare a passi di aggiornamento enormi, e invece non succede: la risposta sta nel flag `--lars`. **LARS** non applica il gradiente così com'è: prima ridimensiona il passo *layer per layer*, in modo che la sua lunghezza dipenda da quanto sono grandi i pesi di quel layer e non da quanto è grande il gradiente. Un gradiente 250 volte più grande quindi **non** produce un passo 250 volte più lungo. Ma LARS aggiusta solo la *lunghezza* del passo, non la sua **direzione**, che resta quella della somma dei due gradienti: passi di dimensione ordinaria, che però puntano tutti verso «resta come l'insegnante». È esattamente il quadro osservato al punto 3 — pesi che crescono di scala senza cambiare direzione.

7. La prova. L'ipotesi era che il colpevole fosse il peso della regolarizzazione, tarato dagli autori per cinque volte le mie epoche: con 500 epoche l'encoder ha il tempo di derivare lentamente dentro il vincolo, con 100 il learning rate si spegne prima e resta fermo dov'era. L'ho verificata rilanciando FD cambiando *solo* λ , da 25 a 5, e lasciando tutto il resto identico (Figura 5). La plasticità dopo il primo task passa da +1 a +7 punti — contro i +13 di FT e gli +11 di PFR — l'accuracy media sale da 41% a 50%, e il metodo si sposta nel piano stabilità-plasticità verso la zona degli altri. Ipotesi confermata: non era un bug, era un iperparametro giusto in un regime e sbagliato in un altro.

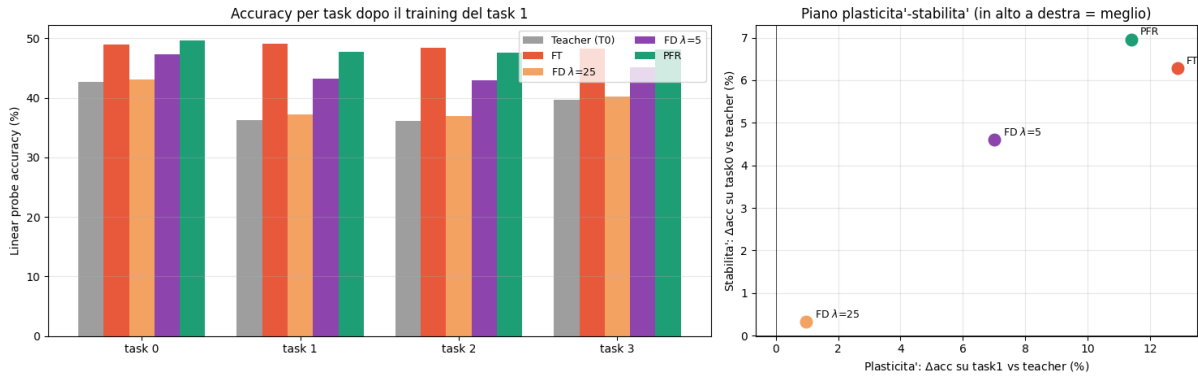


Figura 5. La prova. A sinistra l'accuracy per task dopo il primo task continual, con FD rilanciata a $\lambda = 5$ accanto alle altre. A destra il piano plasticità-stabilità: cambiando solo il peso della distillazione, FD si sposta dall'angolo in basso a sinistra verso la zona di FT e PFR.

Questo collega il progetto a un punto presentato a lezione come problema aperto, il **tuning degli iperparametri nel continual learning**: non si può ritardare un parametro guardando task che non sono ancora arrivati, e un valore corretto in una configurazione non lo è più se se ne cambia un'altra.

Un'avvertenza finale su come si leggono questi numeri, perché all'inizio mi aveva tratto in inganno: nella valutazione task-aware FD ha il forgetting più basso di tutti, praticamente zero. Sembra una virtù e non lo è — è basso perché FD non ha imparato abbastanza da poter dimenticare qualcosa. Accuracy e forgetting vanno letti insieme: preso da solo, l'indicatore di stabilità premia il modello che non fa niente.

9. Conclusioni

Tornando ai tre obiettivi iniziali. La **mappatura tra teoria e codice** è completa: ogni meccanismo del paper è stato individuato nel repository e verificato, e alcune verifiche — i gradienti, l'insegnante congelato, la loss riscritta da zero — sono andate oltre la semplice lettura. Il **revival del codice** è riuscito con tre patch minime e versioni congelate, validate numericamente invece che date per buone. La **riproduzione sperimentale** riesce come pipeline ma non come risultato: il ranking del paper non si ripresenta a 100 epoche per task.

La conclusione più onesta che posso trarre è che **il banco di prova era troppo corto**. A 100 epoche il fenomeno che PFR combatte non si è ancora manifestato — FT semplicemente non dimentica — e senza forgetting da prevenire non c'è modo di distinguere una buona strategia continual da nessuna strategia. Non ho quindi verificato che PFR non funzioni: ho verificato che in questo regime la domanda non è ben posta. FD mostra invece bene il rischio opposto, cioè cosa succede quando una protezione tarata per un altro budget viene applicata a questo: si ottiene stabilità perfetta e apprendimento nullo, che è il caso limite del modello senza plasticità visto a lezione.

C'è anche una lezione metodologica che porto via da questo progetto, e riguarda il rapporto tra un paper e il suo codice. Gli iperparametri pubblicati non sono proprietà del metodo: sono proprietà del metodo *in quel setup*. Ridurre il budget di calcolo di cinque volte e lasciare tutto il resto invariato sembra la semplificazione più innocua possibile, e invece è bastata a ribaltare completamente il risultato di un confronto. Riprodurre un paper non è solo far girare il codice: è capire quali dei numeri scritti nei file di configurazione sono davvero costanti, e quali dipendono da condizioni che non si stanno più rispettando.

Con più tempo di calcolo la strada è chiara: salire a 250 o 500 epoche per task, ripetere su più seed per avere una deviazione standard, completare lo sweep su λ — che a questo punto è diventato l'esperimento più interessante del progetto — e provare il transfer su un dataset diverso, per misurare la qualità delle rappresentazioni fuori dal dominio di training.

Riferimenti

- A. Gomez-Villa, B. Twardowski, L. Yu, A. D. Bagdanov, J. van de Weijer, *Continually Learning Self-Supervised Representations with Projected Functional Regularization*, CVPR Workshop on Continual Learning in Computer Vision, 2022.
- Repository degli autori: https://github.com/alviur/CVPR_PFR (fork del framework CaSSLe).
- F. Giannini, *Continual Learning*, slide del corso di Streaming Data Analytics, Politecnico di Milano.